



Departamento de Informática e Matemática Aplicada – DIMAp

Spring AI – Conversational Memory

Prof. Nélio Cacho

Universidade Federal do Rio Grande do Norte
neliocacho@dimap.ufrn.br

Hora de silenciar o celular...



- Manter o telefone celular sempre desligado/silencioso quando estiver em sala de aula;
- Nunca atender o celular na sala de aula.

Vamos novamente testar a memória do modelo

Olá! Prazer em te conhecer!

Sou um assistente virtual da Pró-Reitoria de Graduação (PROGRAD) da UFRN, criado para ajudar alunos de graduação a esclarecer dúvidas e resolver questões burocráticas relacionadas ao seu curso, matrícula, avaliação, rendimento acadêmico, entre outros temas acadêmicos.

Busco sempre fornecer informações claras baseadas nos regulamentos e normas da UFRN. Caso sua dúvida não esteja coberta pelo regulamento, informarei que não posso responder.

Como posso te ajudar hoje?

Faça sua pergunta:

meu nome é Nélio, qual é meu nome?

Enviar pergunta

Vamos novamente testar a memória do modelo

Resposta à sua Pergunta

Pergunta: meu nome é Nélio, qual é meu nome?

Resposta:

Seu nome é Nélio, conforme informado por você mesmo na pergunta.

Se precisa de informações sobre nome social ou alteração de nome nos registros acadêmicos da UFRN, posso ajudar! Basta perguntar.

Faça outra pergunta:

Enviar

Infelizmente ele não lembra de mim 😞

Resposta da PROGRAD - UFRN

Olá! Não posso responder qual é o seu nome, pois essa informação não está disponível no contexto fornecido e não tenho acesso a dados pessoais dos usuários.

Caso sua dúvida seja sobre nome social ou alteração de nome nos registros acadêmicos da UFRN, posso te orientar sobre esse procedimento.

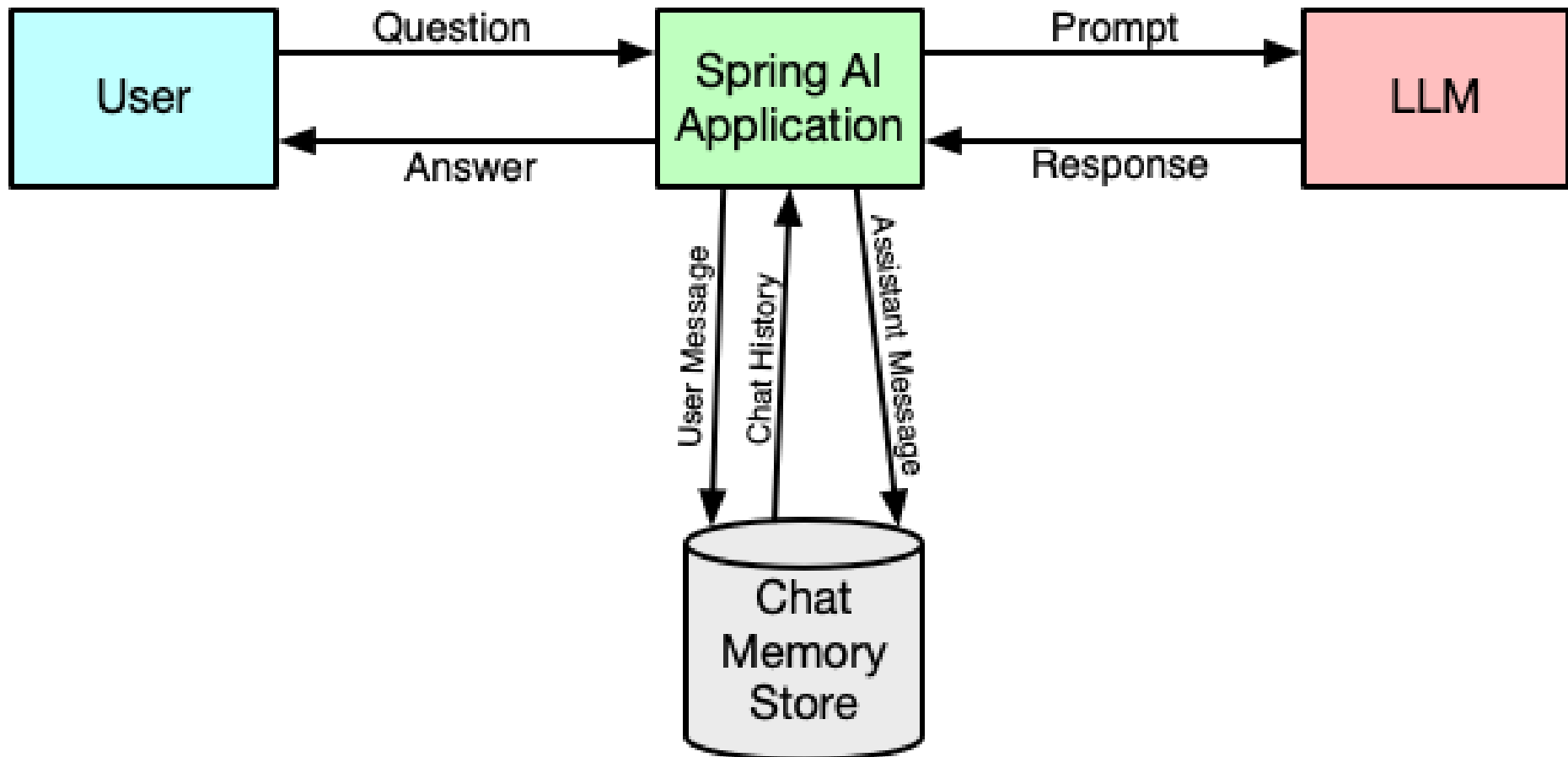
Envie outra dúvida sobre procedimentos acadêmicos da UFRN:

Enviar

LLMs são Stateless por Natureza

- Modelos de linguagem grandes (LLMs) são **stateless** (sem estado), o que significa que eles não retêm informações sobre interações anteriores.
- Isso pode ser uma limitação quando você deseja manter **contexto ou estado** em múltiplas interações.
- Para resolver isso, o Spring AI oferece recursos de **memória de chat ou memória conversacional** que permitem armazenar e recuperar informações em múltiplas interações com o LLM.

Memória conversacional usando Spring AI



Memória conversacional usando Spring AI

- A abstração **ChatMemory** permite implementar vários tipos de memória para suportar diferentes casos de uso.
- O armazenamento subjacente das mensagens é tratado pelo *ChatMemoryRepository*, cuja única responsabilidade é armazenar e recuperar mensagens.
- Cabe à implementação de *ChatMemory* decidir quais mensagens manter e quando removê-las.
- Exemplos de estratégias podem incluir manter as últimas N mensagens, manter mensagens por um certo período de tempo ou manter mensagens até um certo limite de tokens.

Memória conversacional usando Spring AI

- Antes de escolher um tipo de memória, é essencial entender a diferença entre **memória de chat** e **histórico de chat**.
 - **Memória de Chat:** As informações que um modelo de linguagem grande retém e usa para manter a **consciência contextual** ao longo de uma conversa.
 - **Histórico de Chat:** O **histórico completo da conversa**, incluindo todas as mensagens trocadas entre o usuário e o modelo.

Memória conversacional usando Spring AI

- A abstração *ChatMemory* é projetada para gerenciar a **memória de chat**.
- Ela permite armazenar e recuperar mensagens que são relevantes para o contexto da conversa atual.
- No entanto, não é a melhor opção para armazenar o histórico de chat.
- Se você precisar manter um registro completo de todas as mensagens trocadas, deve considerar usar uma abordagem diferente, como depender do Spring Data para armazenamento e recuperação eficientes do histórico completo do chat.

Memória conversacional usando Spring AI

- A Spring AI autoconfigura um bean **ChatMemory** que você pode usar diretamente em sua aplicação.
- Por padrão, ele usa um repositório em memória para armazenar mensagens (*InMemoryChatMemoryRepository*) e uma implementação *MessageWindowChatMemory* para gerenciar o histórico da conversa.
- Se um repositório diferente já estiver configurado (por exemplo, Cassandra, JDBC ou Neo4j), o Spring AI usará esse em vez do padrão.

Implementando Memória Conversacional

- Para implementar memória de chat, é preciso primeiro definir:

```
ChatMemoryRepository chatMemoryRepository = new InMemoryChatMemoryRepository();  
ChatMemory chatMemory = MessageWindowChatMemory.builder()  
    .chatMemoryRepository(chatMemoryRepository)  
    .maxMessages(maxMessages:10)  
    .build();
```

Memória Conversacional

A Spring AI implementa o gerenciamento de memória conversacional como "advisors". Mais especificamente, o Spring AI oferece três advisors diferentes, cada um com sua própria estratégia para gerenciar o histórico do chat:

- **MessageChatMemoryAdvisor**
- **PromptChatMemoryAdvisor**
- **VectorStoreChatMemoryAdvisor**

MessageChatMemoryAdvisor

- **Estratégia:** Adiciona o histórico de chat ao prompt como mensagens distintas (UserMessage, AssistantMessage).
- **Uso:** Ideal para modelos de IA que suportam mensagens baseadas em papéis (e.g., GPT, Gemini).
- **Vantagem:** Mantém a estrutura de conversação clara para o modelo.
- **Limitação:** Não funciona para modelos que não diferenciam papéis nas mensagens.

PromptChatMemoryAdvisor

- **Estratégia:** Injeta o histórico de chat como uma única string formatada dentro de um template de mensagem de sistema.
- **Uso:** Perfeito para modelos que NÃO suportam mensagens baseadas em papéis.
- **Como funciona:** Converte as interações passadas em um bloco de texto que é parte do prompt.
- **Exemplo:** `system_message_template: "O histórico da conversa é: {chat_history}\n{user_input}"`

VectorStoreChatMemoryAdvisor

- **Estratégia:** armazena o histórico em um Vector Store.
- **Funcionamento:** O histórico é transformado em embeddings. Quando uma nova pergunta é feita, o Vector Store é consultado para encontrar trechos do histórico mais semanticamente semelhantes.
- Esses trechos relevantes são injetados no prompt como uma string formatada (assim como no PromptChatMemoryAdvisor).
- **Vantagem:** Escalável para longas conversas, recupera apenas o contexto relevante, otimizando o uso do modelo e custos.

Implementando Memória Conversacional

```
@Override
public String getResposta(String pergunta) {

    return chatClient.prompt()
        .advisors(
            PromptChatMemoryAdvisor.builder(chatMemory).build(),
            QuestionAnswerAdvisor.builder(vectorStore).build())
        .system(systemSpec -> systemSpec
            .text(templatesystem)
            .param(k:"Universidade", v:"UFRN"))
        .user(userSpec -> userSpec
            .text(templateuser)
            .param(k:"Pergunta", pergunta))
        .call()
        .content();
}
```

Ordem dos Advisors no Spring AI

- Como o *QuestionAnswerAdvisor* modifica o prompt original, adicionando texto e contexto adicionais, e como os chat memory advisors do Spring AI armazenam o prompt no histórico, faz diferença qual advisor você adiciona primeiro através do *advisors()*.
- É uma ótima ideia listar o chat memory advisor antes do *QuestionAnswerAdvisor* ao registrar os advisors.
- Fazer isso garante que o prompt armazenado no histórico do chat ainda não tenha sido aumentado pelo *QuestionAnswerAdvisor*.
- No mínimo, isso diminui o tamanho da memória do chat. Mas também é provável que proporcione melhores resultados do histórico do chat.

Acompanhando o conteúdo do prompt

- Para acompanhar o conteúdo do que está sendo passado para o modelo de LLM via Spring AI, use a seguinte configuração:

```
spring:  
  application.name: embedding-service  
  
  ai:  
    chat:  
      observations:  
        log-prompt: true
```

Acompanhando o conteúdo do prompt

← → ↻ ⓘ localhost:8080/consulta?pergunta=meu+nome+é+nelio%2C+qual+é+meu+nome%3F

Seu nome é Nélio. Como posso te ajudar hoje, Nélio?

```
Use the conversation memory from the MEMORY section to provide accurate answers.
```

```
-----  
MEMORY:  
-----
```

```
", "meu nome é nelio, qual é meu nome?.
```

```
Context information is below, surrounded by -----
```

```
-----  
-----
```

```
Given the context and provided history information and not prior knowledge,  
reply to the user comment. If the answer is not in the context, inform  
the user that you can't answer the question.
```

```
"]  
[]
```

Acompanhando o conteúdo do prompt

← → ↻ ⓘ localhost:8080/consulta?pergunta=qual%20é%20meu%20nome?

Seu nome é Nélío. Como posso te ajudar hoje, Nélío?

```
Use the conversation memory from the MEMORY section to provide accurate answers.
```

```
-----
```

```
MEMORY:
```

```
USER:meu nome é nelio, qual é meu nome?.
```

```
ASSISTANT:Seu nome é Nélío. Como posso te ajudar hoje, Nélío?
```

```
-----
```

```
", "qual é meu nome?.
```

```
Context information is below, surrounded by -----
```

```
-----
```

```
-----
```

```
Given the context and provided history information and not prior knowledge,  
reply to the user comment. If the answer is not in the context, inform  
the user that you can't answer the question.
```

```
"]
```

Acompanhando o conteúdo do prompt



localhost:8080/consulta?pergunta=qual+é+meu+nome%3F

Seu nome é Nélio. Como posso te ajudar hoje, Nélio?

Use the conversation memory from the MEMORY section to provide accurate answers.

MEMORY:

USER:meu nome é nelio, qual é meu nome?.

ASSISTANT:Seu nome é Nélio. Como posso te ajudar hoje, Nélio?

USER:qual é meu nome?.

ASSISTANT:Seu nome é Nélio. Como posso te ajudar hoje, Nélio?

", "qual é meu nome?.

Context information is below, surrounded by -----

§2º A Superintendência de Tecnologia da Informação - STI é responsável pela infraestrutura necessária para o armazenamento dos documentos.

Art. 326. A gestão de documentos relativos ao ensino de graduação é responsabilidade das seguintes instâncias acadêmico-administrativas:

I - Pró-reitoria de Graduação - PROGRAD;

II - Departamentos acadêmicos e unidades acadêmicas especializadas: e

Gerenciando Múltiplas Conversas com Spring AI

- Cada interação na Spring AI tem um ID de conversa associado.
- Por padrão, se você não especificar, o ID será "default".
- Isso funciona para um único usuário, mas não para múltiplos.
- Portanto, é preciso manter as conversas de usuários distintos separadas.

Gerenciando Múltiplas Conversas com Spring AI

```
@Override
public String getResposta(String pergunta, String usuario) {
    String conversationId = "conversation-" + usuario;

    return chatClient.prompt()
        .advisors(
            PromptChatMemoryAdvisor.builder(chatMemory).build(),
            QuestionAnswerAdvisor.builder(vectorStore).build()
        ).advisors(advisor -> advisor.param(ChatMemory.CONVERSATION_ID, conversationId))
        .system(systemSpec -> systemSpec
            .text(templatesystem)
            .param(k: "Universidade", v: "UFRN"))
        .user(userSpec -> userSpec
            .text(templateuser)
            .param(k: "Pergunta", pergunta))
        .call()
        .content();
}
```


Acompanhando o conteúdo do prompt

← → ↻ ⓘ localhost:8080/consulta?pergunta=qual%20é%20meu%20nome%20?&usuario=nelio

Seu nome é Nelio. Como posso ajudar você hoje?

```
-----  
MEMORY:  
-----  
  
", "meu nome é Nelio, qual é meu nome?..  
  
Context information is below, surrounded by -----  
  
-----  
-----  
  
Given the context and provided history information and not prior knowledge,  
reply to the user comment. If the answer is not in the context, inform  
the user that you can't answer the question.  
"]
```

Acompanhando o conteúdo do prompt

← → ↻ ⓘ localhost:8080/consulta?pergunta=qual%20é%20meu%20nome%20?&usuario=ricardo

Desculpe, mas não tenho acesso ao seu nome. Como posso ajudar você hoje?

MEMORY:

", "qual é meu nome ?.

Context information is below, surrounded by -----

Given the context and provided history information and not prior knowledge, reply to the user comment. If the answer is not in the context, inform the user that you can't answer the question.

"]
[]

Usando VectorStore para salvar as conversas

```
public class CursosServiceImpl implements CursosService {  
    @Override  
    public String getResposta(String pergunta, String usuario) {  
        String conversationId = "conversation-" + usuario;  
  
        return chatClient.prompt()  
            .advisors(  
                VectorStoreChatMemoryAdvisor.builder(vectorStore).build(),  
                QuestionAnswerAdvisor.builder((vectorStore)).build()  
            ).advisors(advisor -> advisor.param(ChatMemory.CONVERSATION_ID, conversationId))  
            .system(systemSpec -> systemSpec  
                .text(templatesystem)  
                .param(k:"Universidade", v:"UFRN"))  
            .user(userSpec -> userSpec  
                .text(templateuser)  
                .param(k:"Pergunta", pergunta))  
            .call()  
            .content();  
    }  
}
```

Usando VectorStore para salvar as conversas

← → ↻ ⓘ localhost:8080/consulta?pergunta=meu%20nome%20é%20nelio,%20qual%20é%20meu%20nome?&usuario=nelio

Seu nome é Nelio. Como posso te ajudar hoje?

```
Use the long term conversation memory from the LONG_TERM_MEMORY section to provide accurate answers.
```

```
-----  
LONG_TERM_MEMORY:  
-----
```

```
", "meu nome é nelio, qual é meu nome?.
```

```
Context information is below, surrounded by -----
```

```
-----  
meu nome é nelio, qual é meu nome?.  
", "meu nome é nelio, qual é meu nome?.
```

Usando VectorStore para salvar as conversas

← → ↻ ⓘ localhost:8080/consulta?pergunta=qual%20é%20meu%20nome?&usuario=nelio

Seu nome é Nelio. Como posso te ajudar hoje?

Use the long term conversation memory from the LONG_TERM_MEMORY section to provide accurate answers.

```
-----  
LONG_TERM_MEMORY:  
meu nome é nelio, qual é meu nome?.  
Seu nome é Nelio. Como posso te ajudar hoje?
```

```
-----  
", "qual é meu nome?.
```

Context information is below, surrounded by -----

```
-----  
qual é meu nome?.  
meu nome é nelio, qual é meu nome?.  
Seu nome é Nelio. Como posso te ajudar hoje?
```

Usando VectorStore para salvar as conversas

← → ↻ ⓘ localhost:8080/consulta?pergunta=qual%20é%20meu%20nome%20?&usuario=ricardo

Seu nome é Nelio. Como posso te ajudar hoje?

```
-----  
Context information is below, surrounded by -----  
-----  
Context information is below, surrounded by -----  
-----  
qual é meu nome?.  
qual é meu nome ?.  
meu nome é nelio, qual é meu nome?.  
-----  
qual é meu nome?.  
qual é meu nome ?.  
meu nome é nelio, qual é meu nome?.  
qual é meu nome?.  
qual é meu nome ?.  
meu nome é nelio, qual é meu nome?.  
Seu nome é Nelio. Como posso te ajudar hoje?  
qual é meu nome ?.  
meu nome é nelio, qual é meu nome?.
```